FANUC + GUI, explained

Everything I built, with **why** I chose it, **how** it works, and the key lines of real code (read straight from the repo, so the line numbers are right). The full functions are in the long version.

YOUR OPENING

I built the FANUC and the control panel. The FANUC is modelled from FANUC's official description with standard DH and checked to zero error. It's the sorting station: it uses visual servoing and RMRC to pick items off a moving belt, and every move is collision-checked. The GUI controls the whole cell: an e-stop with separate reset and resume, belt speed, item feed, a collision test, and a teach panel to jog the FANUC.

Part A: the FANUC LR Mate 200iD

1. Choosing the robot

Why: an industrial 6-axis arm sized for light sorting of cans and bottles: 717 mm reaches the pick belt and both output belts from one pedestal; 7 kg payload. FANUC publishes an official URDF, so the model uses real data. It meets the spec: 6 DoF, not in the 41013 packs or Corke's toolbox (checked all 47 of its models), not a UR, not a UTS lab robot.

How: joint limits, rated speeds, reach and link lengths come from the URDF and datasheet. It runs at 30% of rated joint speed for safety.

fanuc_lrmate200id/lrmate200id.py lines 56-67

```
 56  QLIM_DEG = [(-170, 170), (-100, 145), (-70, 205), (-190, 190), (-125, 125), (-360, 360)]
 57  QD_MAX_DEG = [450, 380, 520, 550, 545, 1000]   # rated joint speeds, deg/s (datasheet = URDF)
 58  SPEED_SCALE = 0.30                             # fraction of rated speed at 100 % override (was 0.50)
 59  ROT_SPEED = 1.5                                # rad/s, tool re-orientation during straight-line moves
 60  RMRC_W0 = 0.02                                 # manipulability below which RMRC adds damping
 61  RMRC_LAMBDA = 0.05                             # maximum damping (damped least squares)
 62  LINK_RADIUS = 0.05                             # m, arm links as capsules for collision checks
 63  GRIPPER_RADIUS = 0.05                          # m, palm + fingers
 64  REACH = 0.717                                  # m, datasheet
 65
 66  # Link lengths (m), URDF joint origins
 67  D1, A1, A2, A3, D4, D6 = 0.330, 0.050, 0.330, 0.035, 0.335, 0.080
```

2. The DH model

Why: DH is the subject's standard parameter-based model and gives forward kinematics, the Jacobian and IK straight from the toolbox.

How: standard DH: each link is Rz(theta + offset) Tz(d) Tx(a) Rx(alpha). Lengths are the URDF joint offsets. **J2 offset -90 deg** makes q2 = 0 mean 'upper arm vertical' (FANUC's zero), so the URDF limits apply unchanged. **Negative d4, d6:** after the twists those frames' z axes point back to the base. The alphas (-90, 180, -90, 90, -90, 0) give a spherical wrist; it's the same pattern as the Yaskawa GP7.

fanuc_lrmate200id/lrmate200id.py lines 137-145

```
137      def _create_DH():
138          # Joint signs and zeros match the URDF, so QLIM_DEG applies directly.
139          d      = [D1,    0,     0,     -D4,   0,     -D6]
140          a      = [A1,    A2,    A3,     0,    0,      0]
141          alpha  = [-pi/2, pi,   -pi/2,   pi/2, -pi/2,  0]
142          offset = [0,    -pi/2,  0,      0,    0,      0]
143          return [rtb.RevoluteDH(d=d[i], a=a[i], alpha=alpha[i], offset=offset[i],
144                                 qlim=np.radians(QLIM_DEG[i]))
145                  for i in range(6)]
```

SAY THIS

Each row is one joint: rotate theta, move up d, out a, twist alpha. Multiply all six and I get the gripper pose. The -90 offset on joint 2 makes my zero match FANUC's.

3. Proving the model is correct

Why: a model that only looks right in Swift can still be wrong; this is a numerical check.

How: verify_against_urdf rebuilds the arm a second way, straight from the URDF chain, and compares the flange position with my DH model at 200 random joint sets: worst error **0.000 mm**. At q = 0 the flange is at x 0.465, z 0.695 m, matching the URDF.

fanuc_lrmate200id/lrmate200id.py lines 323-331

```
323          rng = np.random.default_rng(seed)
324          worst = 0.0
325          for _ in range(n):
326              q = rng.uniform(self.qlim[0], self.qlim[1])
327              dh = np.eye(4)
328              for link, qi in zip(self.links, q):
329                  dh = dh @ link.A(qi).A
330              worst = max(worst, float(np.linalg.norm(dh[:3, 3] - urdf_flange(q)[:3, 3])))
331          return worst
```

4. Realistic model: meshes, gripper and tool

Why: the spec asks for more realism than the Lab 1 cylinders, and a gripper is needed to pick.

How: real FANUC meshes in FANUC colours, placed by their link frames at q = 0; a parallel-jaw gripper. The tool flips 180 deg (the last DH frame points into the wrist) and moves 10 cm to the TCP, 2 cm above the fingertips. I also fixed a one-step lag: the meshes are now redrawn after q is stored (lines 148-154).

fanuc_lrmate200id/lrmate200id.py lines 112-114

```
112          # The last DH frame's z points back into the wrist; flip it so tool z
113          # points out of the flange, then move out to the gripper's TCP.
114          self.tool = SE3.Rx(pi) * SE3(0, 0, TCP_OFFSET)
```

5. The station and named poses

Why: planning between several known-good poses; the 0.6 m pedestal gave the best IK margin over every pick and place point (0.45-0.75 m tried).

How: the wait pose (above the pick window, 1.07 m up) and a place pose above each output line are solved once at start-up and reused every cycle. _check_reach stops early if a belt is out of reach. The plastic drop is at a yaw of 177 deg but J1 only turns 170 deg, so IK reaches back over the top: a joint-limit consequence, not a bug.

fanuc_cell.py lines 194-197

```
194          y_belt = self.cross.start[1]
195          self.hover_z = BELT_HEIGHT + TALLEST_ITEM + APPROACH
196          self.q_wait = self._solve_above(PICK_WINDOW_X, y_belt, self.hover_z)
197          self.q_place = {m: self._solve_above(*self._place_xy(m), self.hover_z) for m in self.lines}
```

6. Inverse kinematics

Why: works directly from my DH model, robust near singularities, and doesn't flip the arm between configurations.

How: ikine_LM (Levenberg-Marquardt: q += $(J^TJ + \text{lambda } I)^{-1}J^Te$), seeded from the current joints, joint limits on. It raises an error instead of returning a bad answer; sort() catches it and puts the item back on the belt.

fanuc_lrmate200id/lrmate200id.py lines 221-226

```
221      def solve(self, T, q_seed):
222          """IK for TCP pose T, starting from q_seed so the arm keeps the same posture."""
223          sol = self.ikine_LM(T, q0=q_seed, joint_limits=True)
224          if not sol.success:
225              raise RuntimeError(f"LR Mate 200iD cannot reach\n{T}")
226          return sol.q
```

7. Trajectory generation

Why: joint moves for big swings (fast, always reachable); straight-line moves for going up and down near belts, so the gripper never sweeps sideways into a rail.

How: both use a quintic profile $s = 10t^3 - 15t^4 + 6t^5$ (zero velocity and acceleration at both ends). A quintic peaks at 1.875x its average speed, so duration = 1.875 x distance / max speed keeps every joint at or under 30% of rated. Straight lines interpolate the pose and solve IK each step. Every step also passes the collision guard.

fanuc_lrmate200id/lrmate200id.py lines 262-271

```
262      def move_joint(self, q_goal, step, dt):
263          """Quintic joint-space move, timed so no joint exceeds SPEED_SCALE of its
264          rated speed at 100 % (a quintic peaks at 1.875 * travel / duration)."""
265          q0 = np.array(self.q, dtype=float)
266          q_goal = np.array(q_goal, dtype=float)
267          T = np.max(1.875 * np.abs(q_goal - q0) / (self.qd_max * SPEED_SCALE))
268
269          def apply(u):
270              return self._guarded_set(q0 + u * (q_goal - q0))
271          self._run_profile(T, apply, step, dt)
```

SAY THIS

Joint moves for the big swings because they're fast and always reachable; straight lines near the belts. Both are quintic, so they start and stop smoothly, and the 1.875 factor keeps every joint under its speed limit.

8. The camera

Why: the FANUC needs to know what each item is (to sort it) and where it is right now (to catch it moving).

How: each 0.05 s frame: classify new items once; measure position with 1 mm noise; estimate velocity from the change between frames (filtered). If the item isn't in view, it returns nothing, so the arm can't chase stale data.

fanuc_cell.py lines 120-130

```
120          for shape in list(self.world.active_items):
121              p = np.array(shape.T)[:3, 3]
122              if not self.belt.contains(p) or not (self.s_from <= self.belt.progress(p) <= self.s_to):
123                  continue
124              key = id(shape)
125              meas = p + self.rng.normal(0, CAM_NOISE, 3)           # what the camera "sees"
126              if key in self.track:
127                  p_old, v_old, _ = self.track[key]
128                  v = (1 - CAM_VEL_FILTER) * v_old + CAM_VEL_FILTER * (meas - p_old) / DT
129              else:
130                  v = np.zeros(3)
```

9. Visual servoing (the moving pick)

Why: the item never stops, so a fixed IK target would be out of date by the time the arm arrives. Closing the loop on the camera keeps the gripper on it. It's only used where the target moves; fixed targets use IK + quintics.

How: position-based: error = item - gripper from the camera. Command: sideways = item velocity (feed-forward, so no lag) + 8 x error; down at up to 0.3 m/s; turning keeps the gripper pointing down. Grab within 6 mm / 4 mm; give up after 4 s. Tested at belt speeds 0.05-0.25 m/s: every item picked.

fanuc_cell.py lines 308-320

```
308             p_item, v_item = est
309             T = r.fkine(r.q)
310             e_xy = p_item[:2] - T.t[:2]              # visual error: gripper -> item
311             e_z = z_grasp - T.t[2]
312             if np.linalg.norm(e_xy) < SERVO_TOL_XY and abs(e_z) < SERVO_TOL_Z:
313                 break
314             descend = DESCEND_SPEED * max(1.0, self.cross.speed / BELT_SPEED_REF)
315             v = np.zeros(6)
316             v[:2] = v_item[:2] + KP_XY * e_xy
317             v[2] = np.clip(KP_Z * e_z, -descend, descend)
318             v[3:] = KR * self._orientation_error(T.R, R_goal)
319             r.rmrc_step(v, DT)
320             self.step()
```

SAY THIS

Every 0.05 seconds the camera tells me where the item is and how fast it's going. I command the gripper to move at the item's speed plus a correction for the gap while coming down. That's position-based visual servoing.

10. RMRC

Why: the servo gives a velocity, and RMRC follows a velocity directly: smooth joint motion with no IK jumps. Damped least squares keeps it safe near singularities.

How: $q_dot = J^T(JJ^T + lambda^2 I)^{-1}v$, then q += q_dot dt. lambda is 0 unless manipulability drops below 0.02 (near a singularity). Joint speeds are scaled down together if any would exceed 30%, and joints stay in limits. Tested: actual velocity matched the command.

fanuc_lrmate200id/lrmate200id.py lines 290-299

```
290         q = np.array(self.q, dtype=float)
291         J = self.jacob0(q)
292         w = np.sqrt(max(np.linalg.det(J @ J.T), 0.0))           # manipulability
293         lam2 = 0.0 if w >= RMRC_W0 else (1 - (w / RMRC_W0) ** 2) * RMRC_LAMBDA ** 2
294         qd = J.T @ np.linalg.solve(J @ J.T + lam2 * np.eye(6), np.asarray(v, dtype=float))
295         qd_lim = self.qd_max * SPEED_SCALE
296         scale = np.max(np.abs(qd) / qd_lim)
297         if scale > 1.0:                                          # keep the direction, slow it down
298             qd = qd / scale
299         q_new = np.clip(q + qd * dt, self.qlim[0] + 1e-3, self.qlim[1] - 1e-3)
```

SAY THIS

RMRC uses the Jacobian to turn the gripper velocity I want into joint velocities. Damped least squares keeps the joint speeds bounded near a singularity instead of exploding.

11. Collision detection

Why: the tutors said to pick a method and justify it. Line-plane treats links as zero-thickness lines; ellipsoids need per-link fitting. Spheres give links real thickness, include the held item (a bottle hangs about 20 cm below the gripper), and are cheap enough to check every step.

How: 4 spheres (radius 5 cm) along each link between DH frame origins, plus the gripper and held item. An obstacle is a box; a sphere hits it if its centre is inside the box grown by the radius. 30 poses are checked along each planned path. Weakness: small gaps between spheres; fine for these 14-16 cm boxes.

fanuc_lrmate200id/lrmate200id.py lines 207-212

```
207         pts = []
208         for a, b in zip(frames[1:], frames[2:]):         # links 2..6 (link 1 is inside the pedestal top)
209             for s in np.linspace(0, 1, 4):
210                 pts.append((a + s * (b - a), LINK_RADIUS))
211         for s in np.linspace(0, 1, 4):
212             pts.append((flange + s * (tip - flange), GRIPPER_RADIUS))
```

12. Collision avoidance

Why: active avoidance is valued above stopping, and the tutors said waypoints must not be hardcoded. I use a guided search instead of random RRT: I know the obstacle's size, and over the top is the natural way past a box on a conveyor. Trade-off: it only searches upwards.

How: blocked path? Starting 5 cm above the obstacle's top, IK two via poses (above the start and the goal), check all three legs, and raise 5 cm until it's clear (up to 1.45 m). No route: stop and re-check every 0.5 s. If it appears mid-move, the per-step guard freezes the arm, then it replans. Drop spot blocked: use the next spot down the belt. Tested: went over at z 1.14 m; never touched the obstacle.

fanuc_cell.py lines 438-448

```
438         for z in np.arange(ob["hi"][2] + 0.05, VIA_MAX_Z + 1e-6, 0.05):
439             try:
440                 qa = self.robot.solve(LRMate200iD.down_pose(x0, y0, z, self._yaw(x0, y0)), q0)
441                 qb = self.robot.solve(LRMate200iD.down_pose(x1, y1, z, self._yaw(x1, y1)), q_goal)
442             except RuntimeError:
443                 break                                    # can't go any higher
444             legs = [(q0, qa), (qa, qb), (qb, q_goal)]
445             if all(self._path_hits(a, b) is None for a, b in legs):
446                 self.collision_msg = f"Collision predicted: going over the obstacle at z = {z:.2f} m"
447                 print(f"[{self.t:6.1f}s] FANUC: {self.collision_msg}")
448                 return [qa, qb, q_goal]
```

SAY THIS

Before each move I check the planned path. If it's blocked I generate via poses above the obstacle and raise them until the whole route is clear; if there's no route I stop and re-check.

13. The sort cycle

Why: oldest item first (furthest along the belt); modes only switch between cycles, so it never drops a held item.

How: run() waits for the queued item to reach the pick window; sort() does servo pick, place, return. Any failure (IK, servo timeout, lost item) puts the item back on the belt and returns to wait, so the station keeps running.

fanuc_cell.py lines 640-649

```
640             if self.requested_mode == "manual":
641                 self._manual_loop()
642                 continue
643             if self.queue:
644                 shape, material = self.queue[0]      # oldest = furthest along the belt
645                 if float(np.array(shape.T)[0, 3]) <= PICK_WINDOW_X + TRIGGER_AHEAD:
646                     self.queue.pop(0)
647                     self.sort(shape, material)
648                     continue
649             self.step()                              # wait for the next item to reach the pick window
```

Part B: the GUI (control panel)

14. Why tkinter, and how it talks to the robots

Why: Swift only reads its own buttons inside env.step(). During an e-stop the clock is frozen, so a Swift Resume button could never be pressed. tkinter runs independently.

How: the panel runs on the main thread (macOS requires that); each robot runs in its own thread. The GUI never moves a robot or touches Swift: it sets flags or puts commands on a queue.Queue, and only the robot's own thread acts on them, so two threads never move the robot at once.

main_sim.py lines 124-128

```
124     # Control panel on the main thread (Tk needs it on macOS); the watchdog
125     # moves to a background thread. Without a display, fall back to the old loop.
126     threading.Thread(target=watchdog, args=(world, threads), daemon=True, name="watchdog").start()
127     from control_panel import run_panel
128     if not run_panel(world, fanuc, safety):
```

SAY THIS

The GUI is a separate tkinter thread, so it still works during an e-stop when the simulation is frozen. It never moves the robot directly; it queues commands and the robot's own thread executes them.

15. E-stop, reset and resume

Why: the spec: immediate stop, visible state, no restart on release, deliberate reset, then a separate resume, with no busy loop.

How: E-STOP (or Esc) calls world.estop(): running = False, belts stop, reason latched. Every robot thread blocks in tick() at its next 0.05 s step, so everything freezes at once. The bar and every stack light go red. Reset is refused while unsafe (someone inside the fence); Resume is separate. While stopped the threads sleep on a Condition (wait 0.05 s), so there's no busy loop. The button is a Label because macOS ignores button colours.

world.py lines 230-234

```
230         with self._cond:
231             while not self.running:                  # e-stopped: wait, don't spin
232                 self._cond.wait(0.05)
233                 if not self.running:
234                     self._pump_while_stopped()
```

SAY THIS

E-stop freezes the shared clock, so every robot stops at once. It's latched: Reset only acknowledges, and Resume is a separate restart. While stopped the threads sleep on a condition variable, so there's no busy loop.

16. Belt speed and item feed

Why: operator control of the line, and a way to demonstrate a chosen item through the whole system.

How: the slider sets every belt's speed (0.05-0.25 m/s); the FANUC speeds up its descent for faster belts. Item feed: Auto lets the Kawasaki pick whatever is next; Manual makes it finish its current item, then pick only what you choose (it adds one to the tub if needed).

world.py lines 112-116

```
112      def set_belt_speed(self, speed):
113          """Set every belt's speed (m/s). Safe to call from the GUI thread."""
114          with self._cond:
115              for c in self.conveyors.conveyors:
116                  c.speed = float(speed)
```

17. Collision test

Why: the spec requires reacting to an object deliberately placed in a planned robot path.

How: choose 'swing path' or 'metal line drop spot', then Drop or Clear. The swing spot is computed from the FANUC's real path (half-way along the wait-to-drop move). The FANUC thread adds the box and refuses if the arm is already there; the panel shows its decision.

control_panel.py lines 255-257

```
255      def _drop_obstacle(self):
256          self.fanuc.request_obstacle(self.spot.get())
257          self.msg.set(f"Obstacle requested: {self.spot.get()}")
```

18. Teach / jog panel

Why: spec section 4: individual joint jogging, Cartesian x/y/z, and state/fault info. Section 3: move to a final joint state supplied during marking.

How: Auto/Manual switch; in Manual: 6 joint sliders (target, with the actual angle beside each), +/-X/Y/Z with a 5-50 mm step, type 6 angles and Go (planned and collision-checked), gripper, wait pose. Controls are only enabled in Manual. Each control only queues a command.

control_panel.py lines 237-249

```
237      def _cart(self, axis, sign):
238          d = [0.0, 0.0, 0.0]
239          d[axis] = sign * float(self.step_mm.get()) / 1000
240          self.fanuc.jog_cartesian(*d)
241
242      def _goto(self):
243          try:
244              q = np.radians([float(v.get()) for v in self.goto_vars])
245          except ValueError:
246              self.msg.set("Joint state: type six numbers in degrees")
247              return
248          self.fanuc.goto_joints(q)
249          self.msg.set("Go to joint state: " + ", ".join(v.get() for v in self.goto_vars) + " deg")
```

19. Jog safety (the FANUC side)

Why: manual mode must be safe for someone standing at the cell.

How: each 0.05 s step towards the target is limited to 10% joint speed and 250 mm/s TCP (ISO 10218-1 'T1'); before moving, the next pose is checked against a safe-zone box, the belt surface and obstacles, and it stops with a reason if not. Cartesian jogs solve IK for the translated pose first. Measured jog speed 0.22 m/s.

`fanuc_cell.py` lines 604-613

```
604          dq = np.clip(dq, -r.qd_max * T1_JOINT_SCALE * DT, r.qd_max * T1_JOINT_SCALE * DT)
605          moved = float(np.linalg.norm(r.fkine(q + dq).t - r.fkine(q).t))
606          if moved > T1_TCP_SPEED * DT:                # keep the TCP under 250 mm/s
607              dq *= T1_TCP_SPEED * DT / moved
608          why = self._zone_violation(q + dq)
609          if why:
610              self.jog_target = q                      # stop where we are
611              self.jog_msg = f"Stopped: {why}"
612              return
613          r.q = q + dq
```

SAY THIS

When I click +Z, the GUI only queues a command. The FANUC thread solves IK, checks the new pose is in the safe zone and clear of obstacles, then moves at reduced speed, at most 250 millimetres a second.

20. Live state and faults

Why: the spec asks for useful state or fault information. Tk isn't thread-safe, so the panel reads robot state rather than robot threads touching widgets.

How: root.after re-runs _refresh every 100 ms: cell state (running / e-stop / reset), FANUC mode, TCP, joint angles, and red fault text (e-stop reason, jog refusals, collision decisions).

`control_panel.py` lines 294-298

```
294      def _refresh(self):
295          try:
296              self._refresh_inner()
297          finally:
298              self.root.after(REFRESH_MS, self._refresh)
```

21. Safety equipment and sensors

Why: placed by risk: the gate is the only way into the fenced cell, so the curtain and speed zone go there.

How: fence + gate; light curtain at the gate (ISO 13855: S = 1.6 x 0.25 + 0.128 = 0.53 m minimum; the nearest hazard is 0.97 m in), which trips the e-stop; a speed zone in front that slows the whole cell to 25%; red/green lights and an e-stop at every robot. Swift can't recolour shapes, so the lit lamps are swapped in and out.

`safety.py` lines 269-277

```
269          blocked = p.front >= GATE_Y and p.back <= GATE_Y
270          if blocked != self.curtain_blocked:
271              self.curtain_blocked = blocked
272              self._show_beams(blocked)
273              if blocked:
274                  self.trips += 1
275                  self.world.estop("Light curtain broken at the cell gate")
276          self.in_warning_zone = p.front >= GATE_Y - WARN_DEPTH and p.back <= GATE_Y
277          self.world.slow_factor = WARN_SPEED if self.in_warning_zone else 1.0
```

HONEST LIMITATIONS: SAY IF ASKED

The camera reads sim positions plus noise; collision spheres are conservative; avoidance only searches upwards (an RRT would also go round). The physical e-stop and the real robot are the group's; a PLC mushroom e-stop would call the same world.estop().